# Index of Malware Analysis


| No. | Topic of The Report |
|---|---|
| 1. | Introduction |
| 2. | What is Malware |
| 3. | Type of Malware |
| 4. | Static vs. Dynamic Malware Analysis |
| 5. | PE File Structure Basics |
| 6. | Common Malware Analysis Tools |
| 7. | Detect It Easy (DiE) Overview |
| 8. | Detect It Easy (DiE) Features |
| 9. | How to Install Detect It Easy |
| 10. | How to Analysis File using Detect It Easy |
| 11. | Conclusion Of this Report |

# Malware Analysis Report

## ❖ Introduction:

- Malware (malicious software) refers to any program designed to harm computers, steal data, or disrupt services. In this report, we explore the theory behind malware analysis for a cybersecurity Purpose. We define what malware is, explain common types (like viruses, worms, trojans, etc.), and compare two main analysis approaches: static analysis (examining code without running it) and dynamic analysis. The report also introduces two popular analysis tools: Detect It Easy, and Process Monitor, a Windows utility for monitoring real-time activity (dynamic analysis).By studying these concepts, a student gains a solid foundation in how cybersecurity professionals detect and investigate malicious software.

## ❖ What is Malware?

- Malware is short for "malicious software." It is software intentionally created to infiltrate, damage, or take control of computer systems. Attackers write malware to steal personal information, spy on users, use computers without permission, or hold systems hostage. For example, malware can lock your files with encryption (as in ransomware) or secretly send your passwords to a hacker. According to Microsoft, malware is any software designed to "disrupt, damage, or gain unauthorized access" to systems. Cisco similarly defines malware as intrusive code built to steal data or destroy systems.

## ❖ Types of Malware:

- **Virus:** A virus is code that attaches itself to a legitimate program or file and runs when that program starts. It can replicate by infecting other files. Viruses often corrupt data or damage systems when activated. For example, a macro virus might hide in a document and infect other documents when opened.

- **Worm:** A worm is standalone malware that replicates itself and spreads over networks. Unlike a virus, it does not need a host program. Worms exploit software vulnerabilities or use removable drives to jump to new machines. Once inside a network, a worm can slow connections or carry additional attacks (like launching ransomware).

- **Trojan:** A Trojan (or Trojan horse) disguises itself as a useful or benign program. Users are tricked into running it (for example, thinking it's a game or update), and the Trojan then installs backdoors or spyware. Trojans do not self-replicate, but they can deliver serious payloads once executed.

- **Ransomware:** Ransomware is malware that encrypts a victim's files or system and demands payment to unlock them. The attacker typically hides the encryption key and promises to give it back if the victim pays a "ransom." In many cases the encrypted data becomes inaccessible. Cisco explains that ransomware "gains access to sensitive information, encrypts it so the user cannot access it, and then demands a financial payout for the data to be released".

- **Spyware:** Spyware secretly monitors user activity and reports it to a remote attacker. It might track websites visited, record keystrokes, or copy files without the user knowing. The goal of spyware is usually to steal personal or financial information. For example, a keylogger is a type of spyware that logs all keystrokes to capture passwords and credit card numbers.

- **Adware:** Adware automatically displays or downloads advertisements on a user's device. Sometimes it also tracks browsing habits. Although adware can be annoying and slow down a system, it is not always dangerous. However, some adware bundles more malicious code. Cisco notes that adware can redirect browsers to unsafe sites and "even contain Trojan horses and spyware" if not properly controlled.

- **Rootkit:** A rootkit is software that hides deep in a system to give attackers "root" (administrator) control. It can modify core OS components or firmware so that the malware stays hidden from normal tools. CrowdStrike describes a rootkit as "software designed to give malicious actors control of a computer, network or application," often by setting up backdoors and evading detection. Rootkits are dangerous because they allow malware to persist undetected for a long time.

- **Bot and Botnet:** A bot is a program that runs automated tasks, often connecting to a central server. A botnet is a network of many infected machines (bots) controlled together. Cybercriminals use botnets to launch large-scale attacks: for example, flooding a website with traffic (DDoS attack) or sending spam emails. CrowdStrike explains that bots in a botnet work under an attacker's control and can carry out coordinated attacks.

- **Keylogger**: A keylogger is a form of spyware that records every keystroke on the keyboard. It can be hardware or software based. Malicious keyloggers capture passwords, messages, and other sensitive input. According to CrowdStrike, keyloggers can be used to steal banking info or spy on business communications.

- **Fileless Malware:** Fileless malware resides in a computer's memory (RAM) rather than on the hard drive. Since it does not create or use actual files, it is very stealthy and hard for antivirus software to detect. Cisco defines fileless malware as memory-resident code that "makes changes to files that are native to the operating system" and thus leaves little trace. An example is malware that exploits built-in tools (like PowerShell) to run malicious commands without dropping new files.

- **Logicbomb:** A logic bomb is harmful code hidden in a program. It stays inactiveuntil a specific condition is met, then it causes damage like deleting files ordisrupting systems. It does not spread like other malware.

- **Scareware:** Scareware is a type of malware that scares users into thinking their device has a virus. It shows fake warning messages and pushes them to downloadfake security software, which can steal personal data, money, or install more malware.

- **Polymorphic:** Polymorphic malware is harmful software that keeps changing its code to avoid detection. Because it constantly modifies itself, traditional antivirus programs that look for fixed patterns cannot easily detect it.

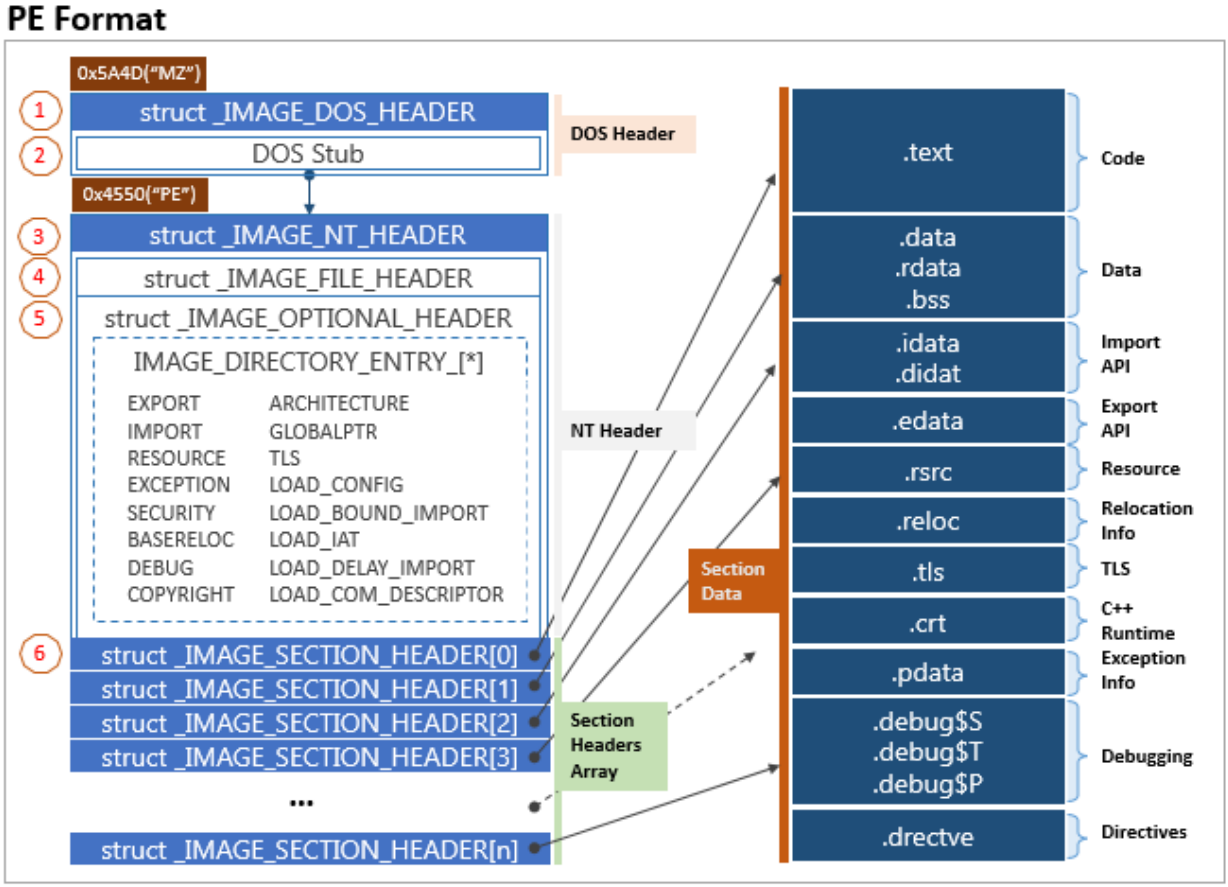
- **Cryptojacking:** Cryptojacking malware secretly uses a computer's CPU or GPU to mine cryptocurrency without permission. It slows down the system, causes overheating, and increases electricity costs, while attackers make money using the victim's resources.

❖ **Static vs. Dynamic Malware Analysis:**

- **Static Analysis:** In static analysis, we examine the malware file (its code, structure, and metadata) without actually running it. This often involves looking at the binary in a disassembler or decompiler, reading text strings embedded in it, and checking file properties. Static analysis is typically quick and can even be automated. For example, tools can scan a file's digital fingerprint (hash or byte patterns) against a database of known malware signatures. The static approach helps identify malware families and some capabilities right away. However, it can miss behaviors that only appear at runtime. If malware is obfuscated or only activates under certain conditions, static analysis might not reveal its full story
- Here we can use Tool like Virustotal to scan a file or documents
- **Link :- https://www.virustotal.com/gui/home/upload**

- **Dynamic Analysis:** Dynamic analysis means running the malware in a controlled environment (like a sandbox or virtual machine) and observing what it does. Here, the analyst watches the malware execute, tracking its behavior in real time. This is more thorough because you can see every system call, file change, or network request the malware makes. For instance, while the malware runs, a monitoring tool can log which files are created or deleted, which registry keys are modified, and what network addresses the code contacts. Dynamic analysis is often behavior-based: it can detect new or unknown malware by the actions it takes (whereas static analysis relied on existing signatures). The downside is that it requires a safe environment (to avoid infecting real systems) and can take longer. Dynamic analysis tools also risk being evaded by malware that detects it's in a virtual machine or sandbox.
- Here we can use Tool like Process Monitor to scan or analysis system.
- **Link:- https://learn.microsoft.com/en-us/sysinternals/downloads/procmon**

❖ **PE File Structure Basics:**



- Nearly all Windows executables and DLLs use the PE (Portable Executable) format. The PE file is organized into a series of headers and sections that tell the operating system how to load and run the program. The main parts are:

- **DOS Header and Stub:** The file begins with the "MZ()" magic bytes. An old DOS stub follows that typically prints "This program cannot be run in DOS mode" on older PCs.

- **PE Header (IMAGE_NT_HEADERS):** Starts with "PE\0\0". This includes the COFF File Header (machine type, number of sections, etc.) and the Optional Header (which is required). The Optional Header contains things like the entry point, image base, and the Data Directory (pointers to imports, exports, resources, etc.).

- **Section Table:** A list of sections follows the headers. Each section has a name (up to 8 characters) and data. Common section names include:
  - **.text** – executable code and the entry point.
  - **.rdata/.idata** – import table (DLL/API usage).
  - **.data** – initialized global data.
  - **.rsrc** – resources (icons, dialogs, strings).
  - **.bss** – uninitialized data (does not occupy file space).
  - **.reloc** – relocation information for loading at non-preferred address.
  - **.debug, .tls, .pdata, etc.** – for debug info, thread-local data, exception handling, etc.

- The loader reads these headers to map each section into memory with proper permissions (e.g. .text is RX, .data is RW). By studying the PE structure, analysts can find useful information: for instance, the Import Table shows which system libraries and functions the program uses (networking, encryption, etc.). The presence of unusual sections (like "UPX0"/"UPX1" from UPX packer) or very high entropy in a section can indicate packed malware. Understanding the PE layout is fundamental to malware analysis.

❖ **Common Malware Analysis Tools:**

- **Process Monitor (ProcMon):** A dynamic analysis tool from Microsoft's Sysinternals suite. It logs all real-time file, registry, and process activity on a Windows system. Analysts use ProcMon to see exactly what a piece of malware does (for example, creating files or modifying registry keys) when it runs, helping to trace its behavior and impact.

- **Wireshark:** A network packet analyzer used for dynamic analysis. It captures and inspects network traffic across many protocols. Malware analysts use Wireshark to view the data a malicious program sends or receives (such as command-and-control messages or downloaded files), revealing how the malware communicates with the outside world.

- **Ghidra:** A static analysis and reverse-engineering tool (disassembler/decompiler) developed by the NSA. Ghidra lets analysts examine a program's assembly code without running it. Its built-in decompiler can convert low-level machine instructions into a higher-level, human-readable form, making it easier to understand the logic and intent of malware in a controlled, non-executable environment.

- **YARA:** A static, pattern-matching tool for detecting malware. Analysts write "rules" with specific text or byte patterns that describe malware signatures. YARA is widely used to identify known malicious files (binaries, scripts, or memory dumps) by matching these patterns. It's useful for quickly scanning lots of files or memory to flag anything matching known malware indicators.

- **Cuckoo Sandbox:** An automated dynamic analysis system. It runs suspicious files inside an isolated virtual machine and records everything the malware does. Cuckoo captures detailed results like files created, registry changes, network traffic, and even screenshots. This automated sandboxing helps analysts safely see the full behavior of a malware sample during execution.

- Each of these tools helps malware analysts uncover different aspects of a malicious program's behavior or structure, making them valuable in both student projects and real-world investigations

- **Detect It Easy (DiE) Overview:**
  - Detect It Easy (DiE) is a lightweight static analysis tool for identifying file types and uncovering packers/protectors in executables.
  - It works by scanning the file against a signature database (and heuristics) to report what it finds.
  - **File Type Detection:** DiE can recognize many formats on Windows, Linux, macOS, Android, etc.
  - **Packer/Protection Identification:** DiE has signatures for hundreds of known packers and protectors (e.g. UPX, Aspack, Themida, .NET Reactor).

- **Detect It Easy (DiE) Features:**
  - **Signature-Based and Heuristic Scanning:** DiE matches file bytes against a library of signatures for file formats, packers, compilers, etc.. It also uses heuristic checks for unknown formats.

  - **Supported Formats:** PE, ELF, APK, IPA, JAR, ZIP, ISO, Mach-O, DEX, and many more.

  - **Packers/Protectors:** Identifies known packers or obfuscators (UPX, ASProtect, Themida, vmProtect, etc.).

  - **Entropy / Packing Clues:** Packed files usually have high entropy. While DiE doesn't show an entropy meter, detecting a packer (or lack of normal section names) suggests packing.

  - **Compiler/Linker Info:** Reports if it detects the compiler or linker (e.g. Microsoft Linker, Borland).

  - **Strings Search with Regex:** Built-in viewer allows searching strings and using regex (e.g. find all "http" occurrences).

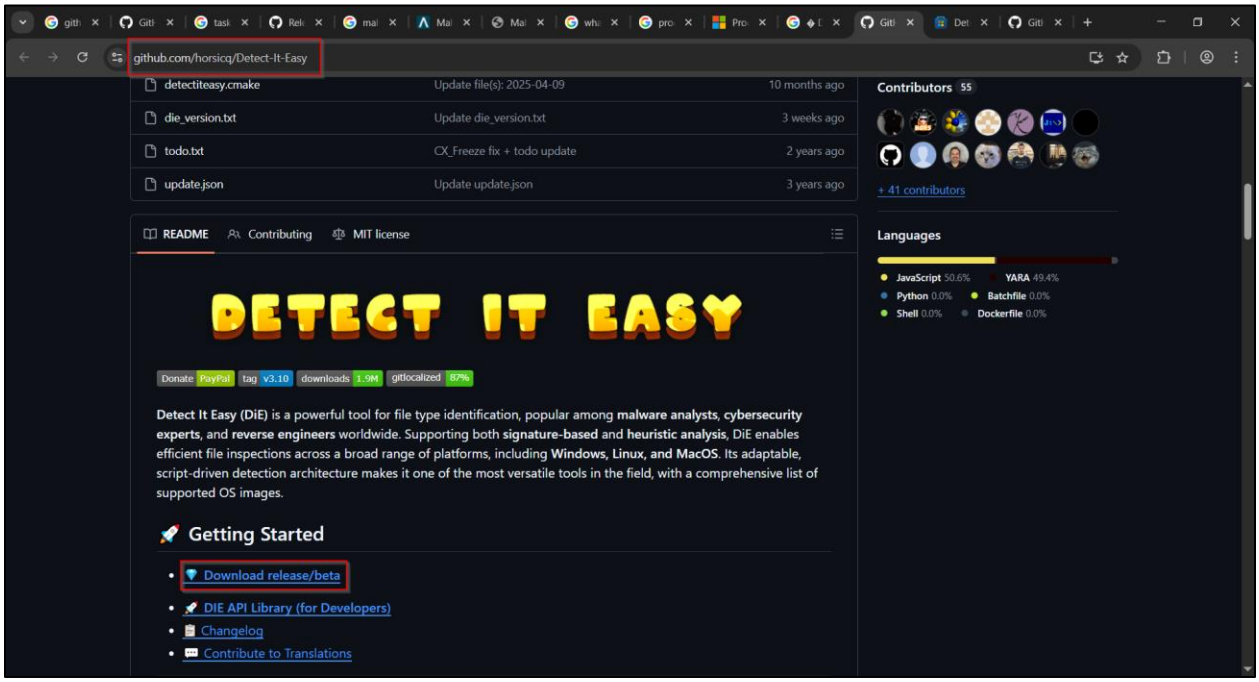
  - **Hash Computation:** Can compute file hashes (MD5/SHA256) which are often used as IOCs.

  - **Import/Export Info:** Shows libraries and functions imported and exported (helpful to guess functionality).

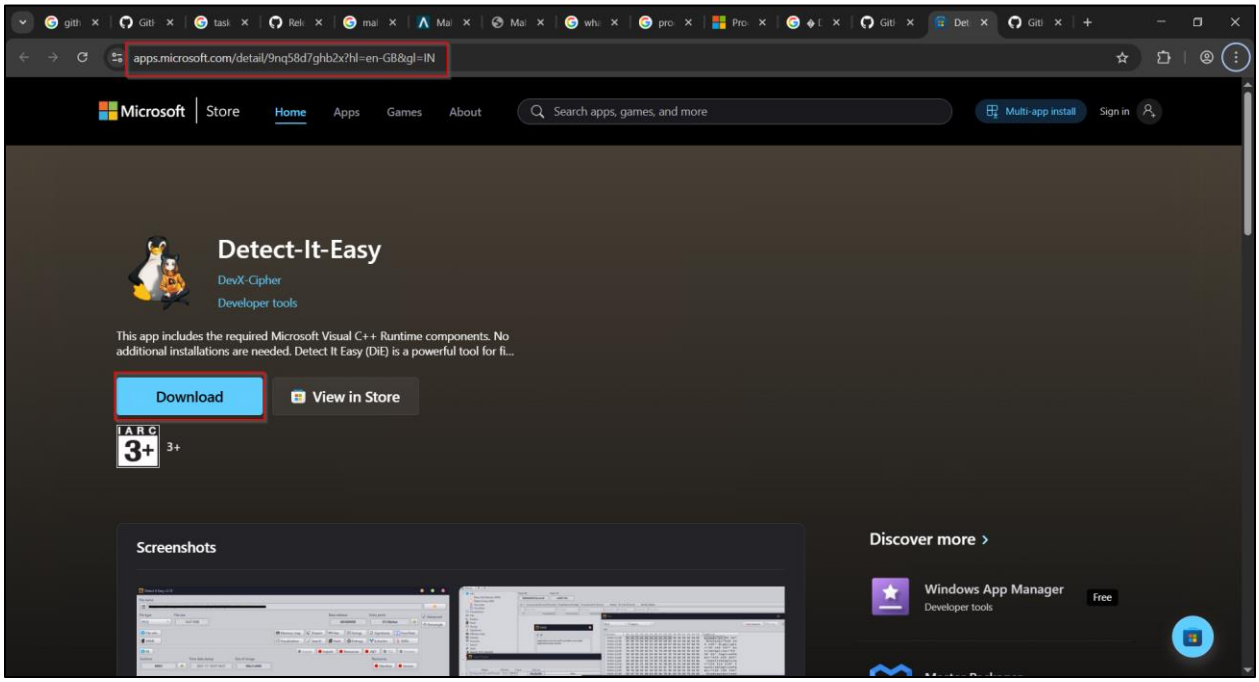
  - **VirusTotal Integration:** For convenience, DiE can query VirusTotal if online (shows VT scan results) (useful but optional).

❖ **How to Install Detect It Easy?**

- Here click this link 1:- https://github.com/horsicq/Detect-It-Easy you can download using this link.

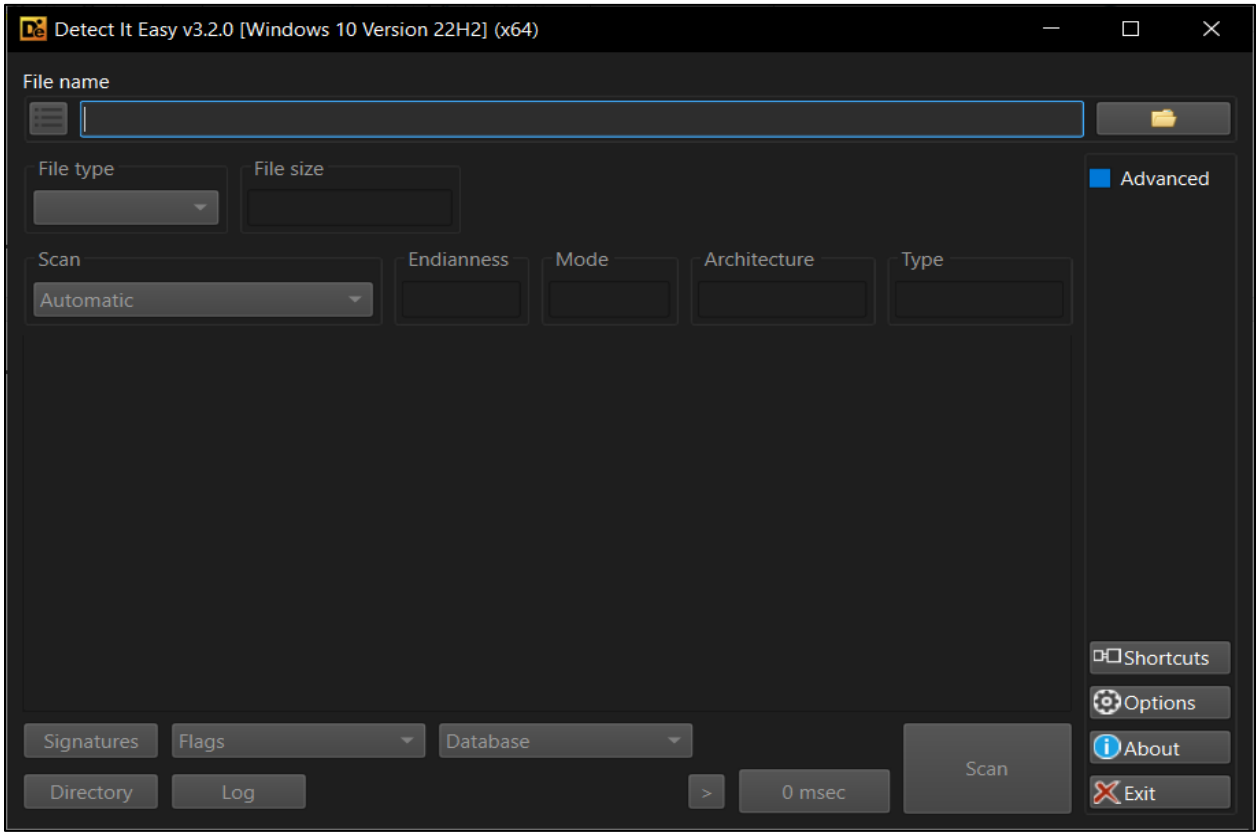



- **Link 2:-** https://apps.microsoft.com/detail/9nq58d7ghb2x?hl=en-GB&gl=IN This is Microsoft store also has this tool.

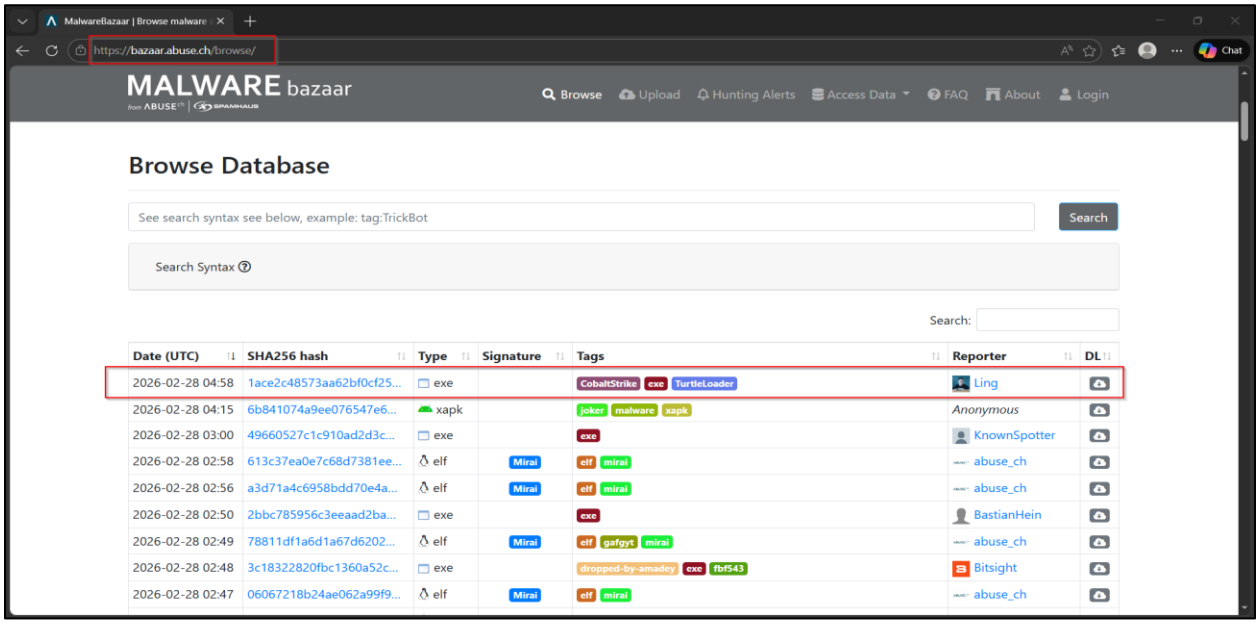


## ❖ How to Analysis File using Detect It Easy:

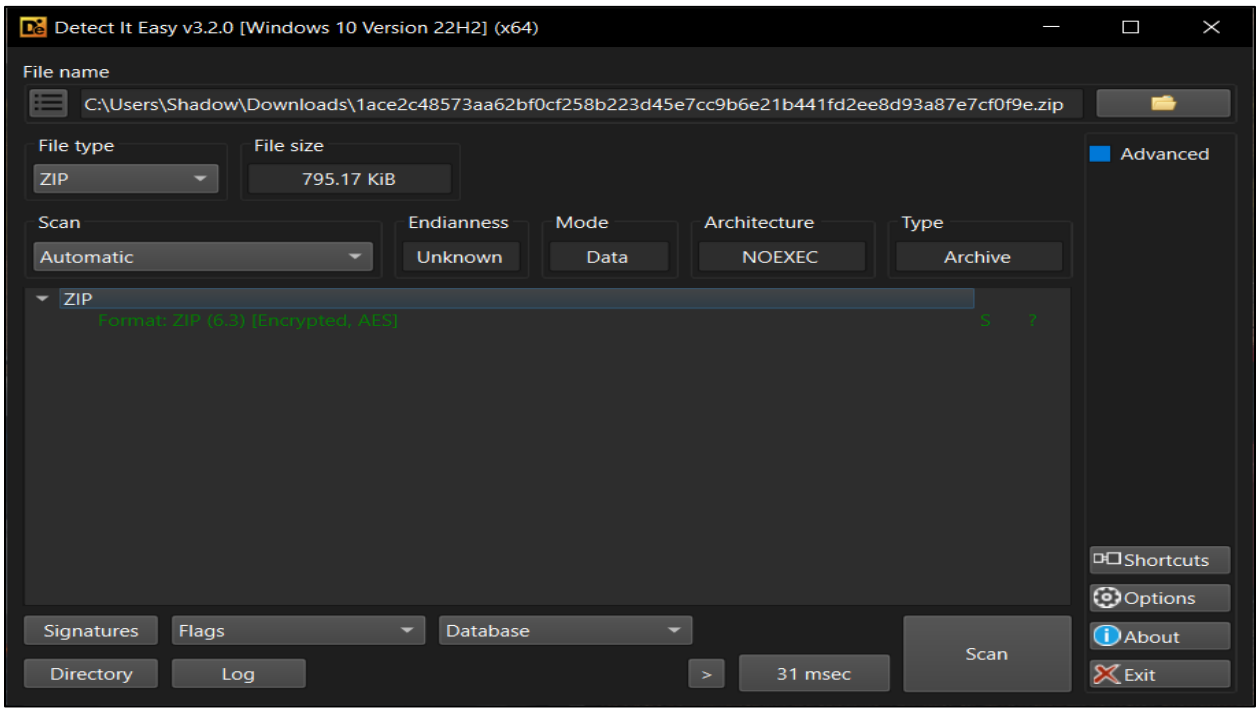
- Here is the interface of our tool Detect It Easy.



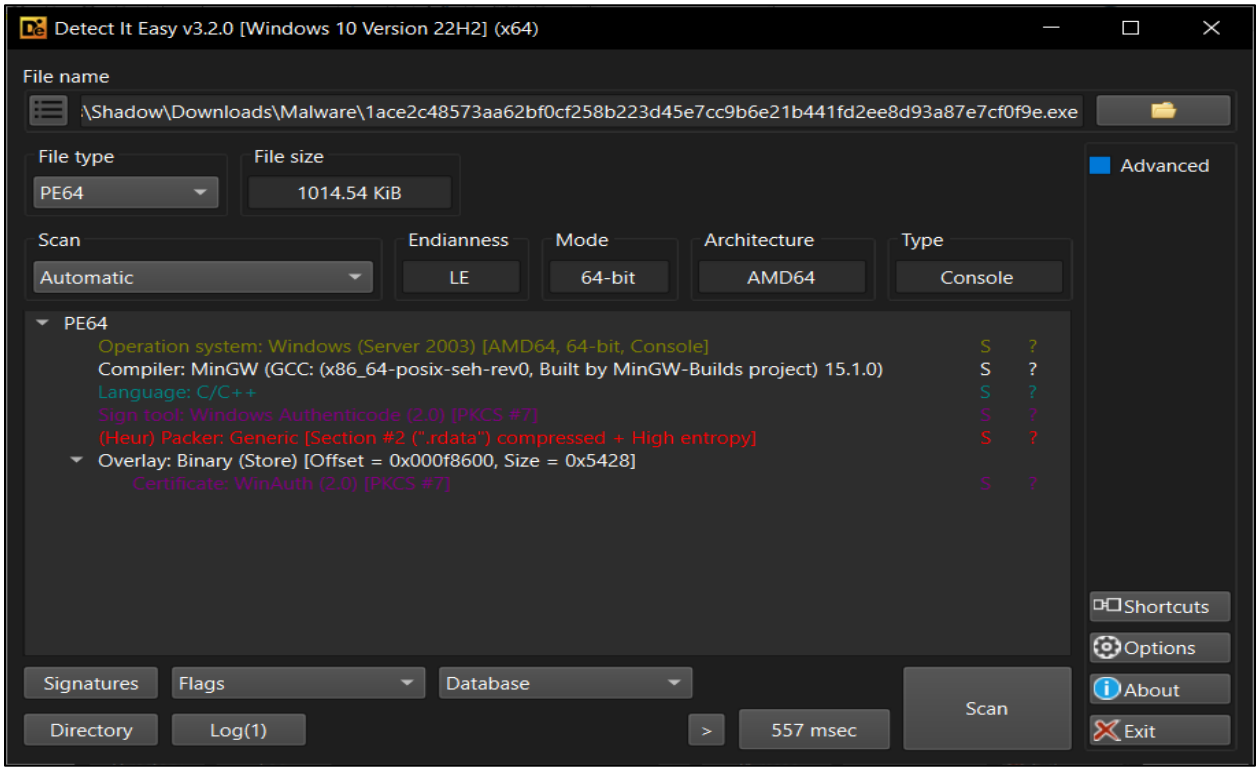

- Here I use a websites called a Malware bazaar to analysis using our tool.



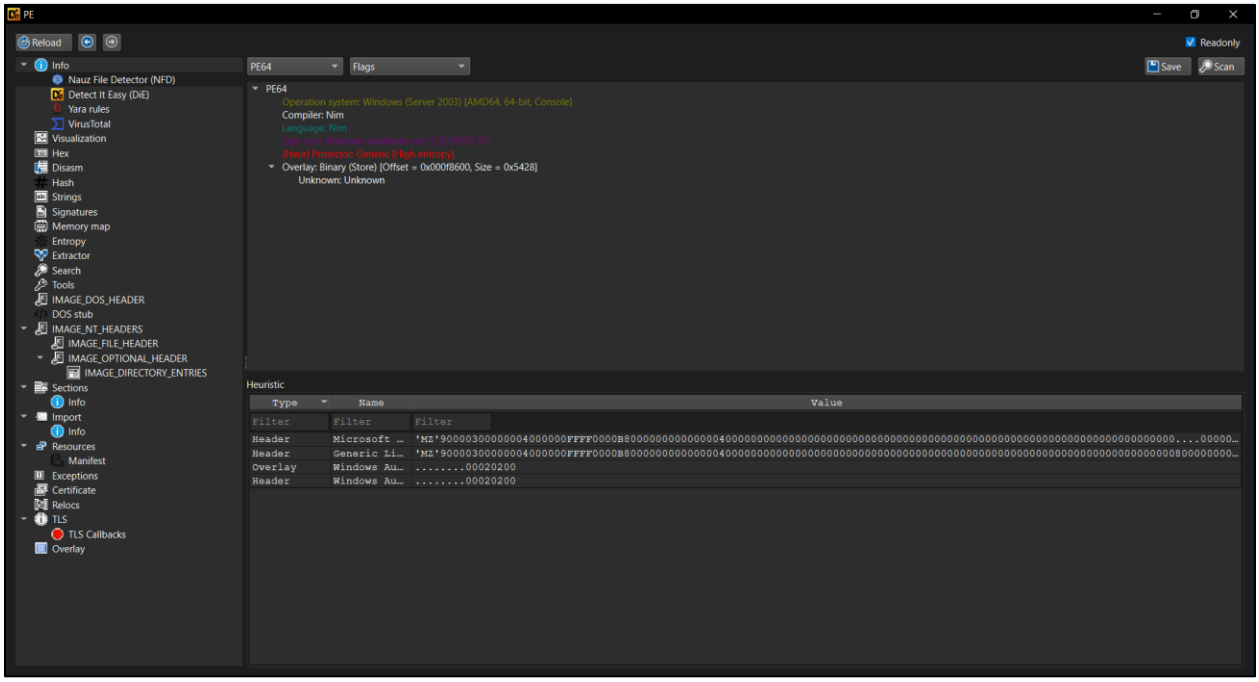
- Simple just drag and drop the zip file into our tool. You can see here our file is Encrypted.



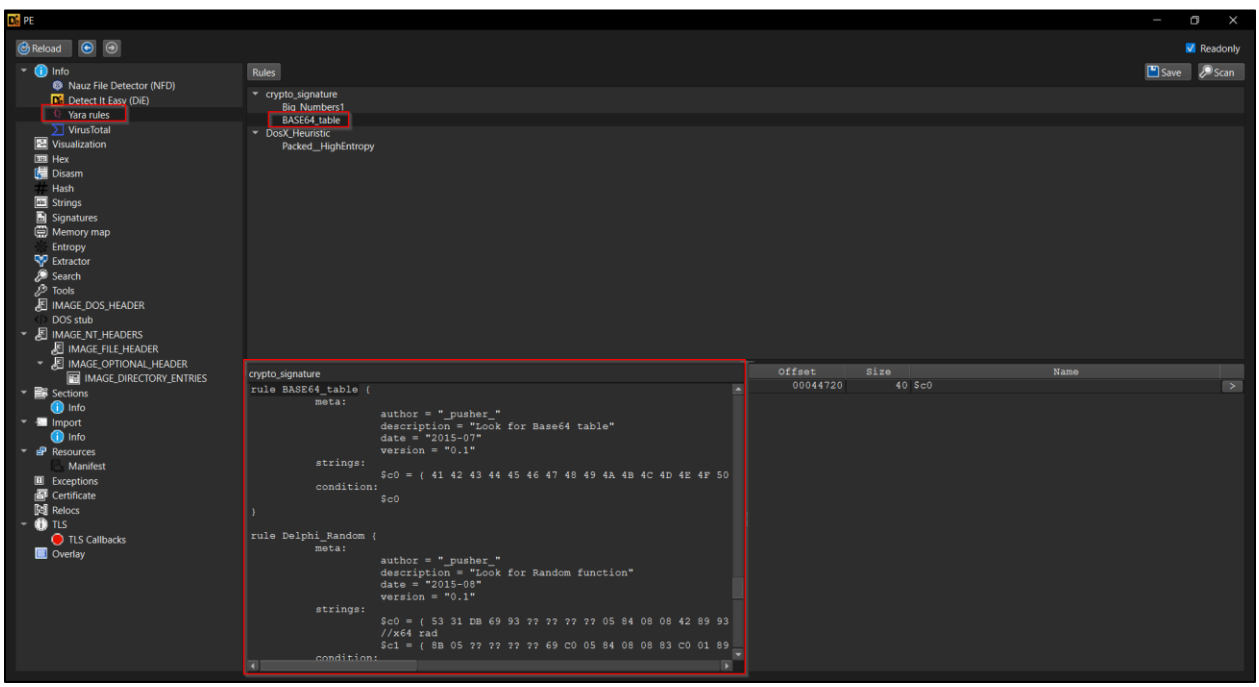

- Or you can just unzip and then past the our exe file into the tool.



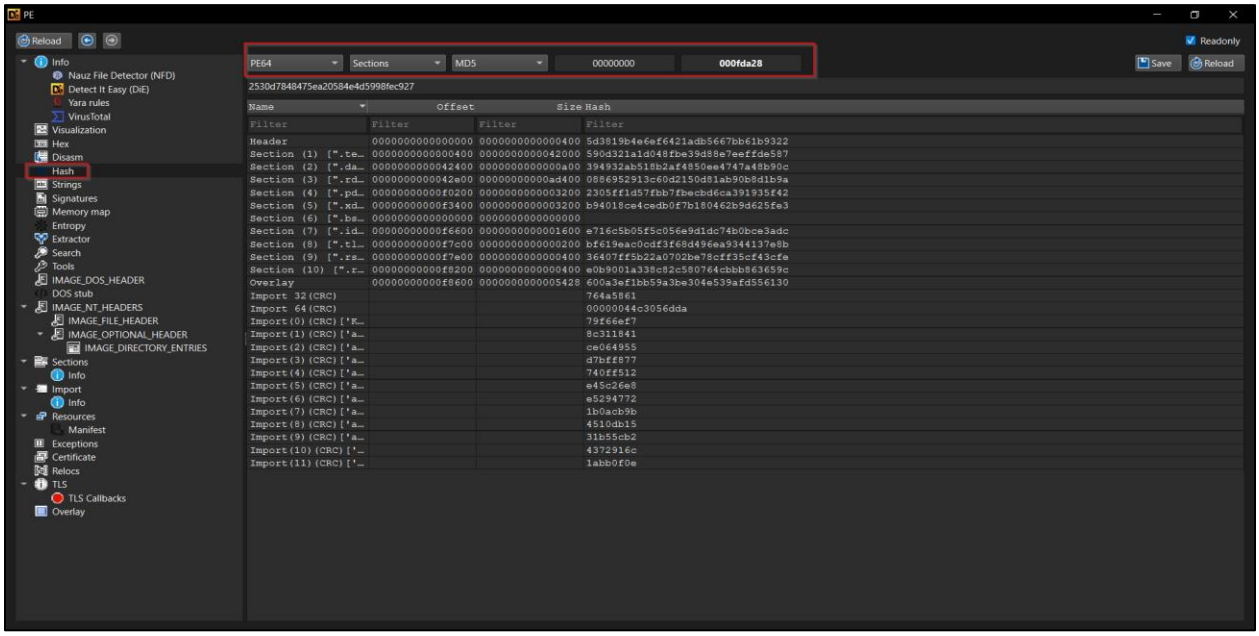
- As a scan perform the option shows up advanced click there and you see all options as per screen shot there is completely info about you exe file.



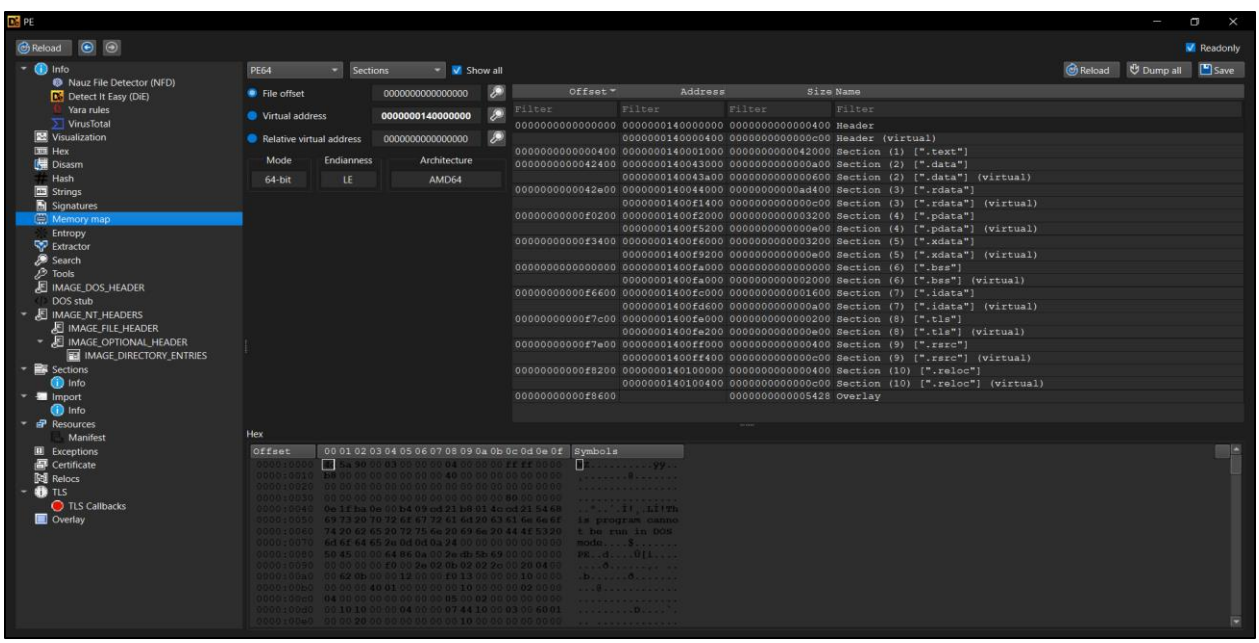

- There is option of Yara rules and that is for text-based, code-level pattern-matching definitions used by security analysts to identify, classify, and detect malware families by describing their unique characteristics.



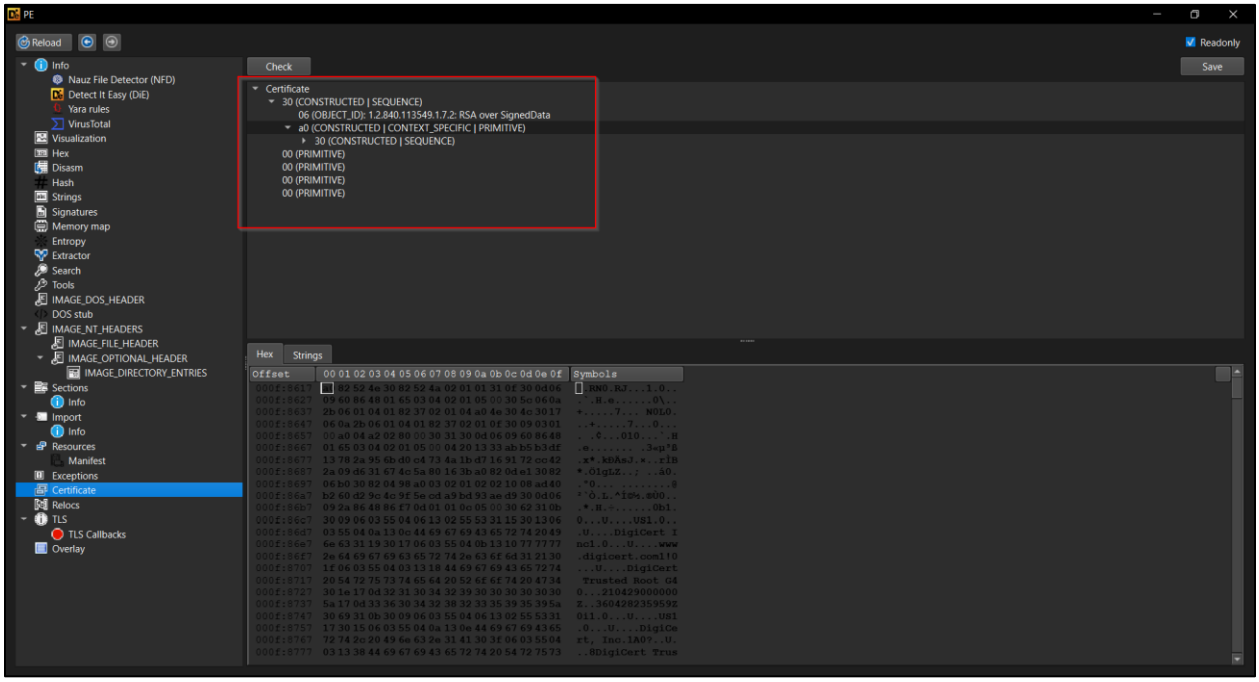
- Here is hash options that use for a a unique, fixed-length string of alphanumeric characters generated by a formula (hash function) based on the contents of a file. It acts as a digital fingerprint, enabling verification of data integrity; if even one bit of the file changes, the hash changes completely.



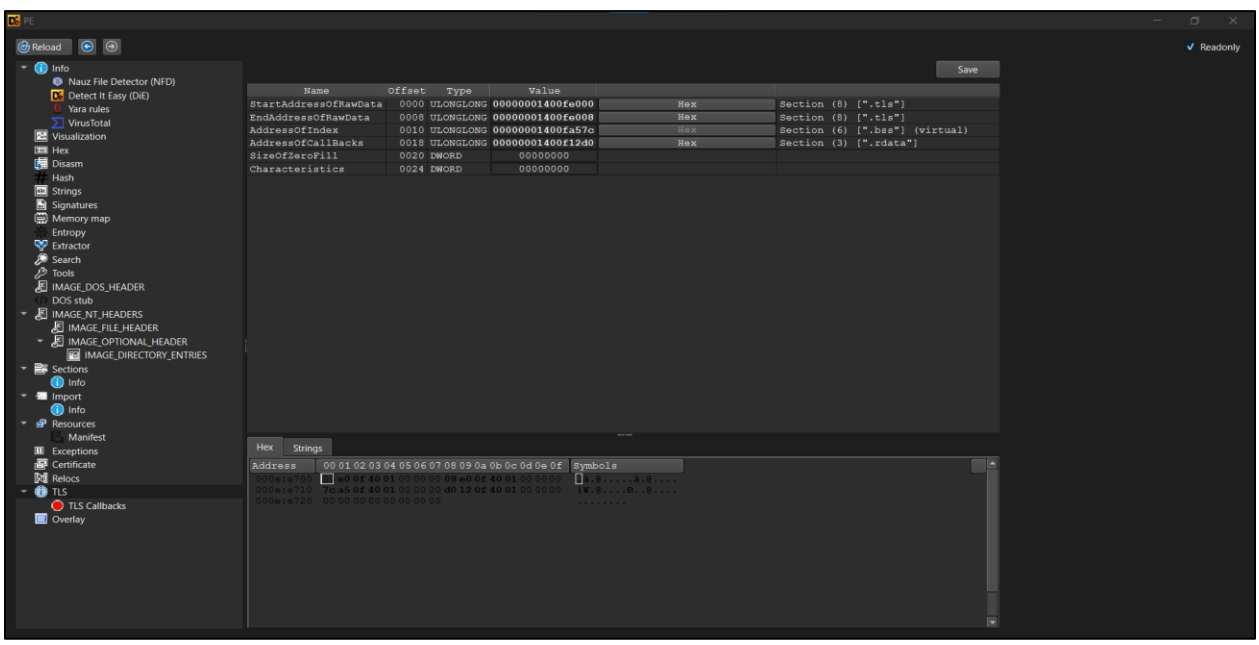
- Here we have Memory map a technique that maps a file's contents directly into a program's virtual address space, allowing the file to be accessed like an array in RAM rather than using slow read()/write() system calls. The OS manages data transfer between disk and memory via demand paging, improving performance and allowing shared memory access.



- We can even check our exe file Certificate to identify what is problem or it is same key as we give me or not.




- Here is TLS in our exe file for a cryptographic protocol that secures data transmitted over networks, ensuring privacy, data integrity, and authentication. It is the successor to SSL, widely used to encrypt web traffic (HTTPS), email, and file transfers (FTPS) to prevent eavesdropping and tampering.



## ❖ Conclusion:

In this project we conducted static analysis of a malware sample using Detect It Easy (DiE), identifying its file type, architecture, and embedded string artifacts. Detect It Easy is known to be a "powerful tool for file type identification" that is widely used by malware analysts. We also reviewed malware categories and examined the Windows PE format – understanding PE is fundamental in static analysis, since "the PE format is one of the most important concepts in understanding any malware binary". From the binary we extracted indicators of compromise (IOCs) such as suspicious URLs and hashes; an IOC can be thought of as "forensic data that identifies potentially malicious activity". Other tools (for example, Ghidra for reverse-engineering and Wireshark for network capture) were noted as useful complements for deeper analysis.

Overall, this hands-on work showed that static analysis with tools like DiE is fast and safe: as one source summarizes, "static analysis provides quick and efficient results by examining the malware's code and structure". Practice with these methods – file-format inspection, IOC extraction, and static reverse-engineering – builds a strong foundation in malware reverse-engineering and cybersecurity. In short, learning to use DiE and understand PE files and IOCs strengthens core skills for threat detection and response, making future malware investigations more effective.

❖ **References:**


- https://github.com/horsicq/Detect-It-Easy
- https://horsicq.github.io/
- https://learn.microsoft.com/en-us/security/
- https://www.kaspersky.com/resource-center/threats/malware
- https://www.crowdstrike.com/cybersecurity-101/malware/malware-analysis/
- https://www.paloaltonetworks.com/cyberpedia/indicators-of-compromise-ioc
- https://en.wikipedia.org/wiki/Portable_Executable
- https://learn.microsoft.com/en-us/sysinternals/downloads/procmon